

Every Camera

Программа управления камерами Canon (gphoto2), SPTT (CSDU-429), Infra (Tanho SW1300 SWIR) и Sentry (Princeton Instruments CCD via imagerd_rt).

Два режима работы: консольный (headless) и графический (PyQt5).

Отдельные программы для наблюдателя:

- ``monitor_app.py`` — мониторинг всех запущенных камер;
- ``viewer_app.py`` — просмотрщик кадров: по локальной сети любой снятый кадр, через HiveMQ — последний;
- ``focus_app.py`` — помощник настройки резкости: живой поток с изменяемыми параметрами (только по локальной сети).

Структура проекта

<code>main.py</code>	— точка входа: измерения (консоль / GUI)
<code>monitor_app.py</code>	— точка входа: мониторинг (отдельная программа)
<code>viewer_app.py</code>	— точка входа: просмотрщик кадров (наблюдатель)
<code>focus_app.py</code>	— точка входа: настройка резкости (наблюдатель)
<code>setup_app.py</code>	— мастер настройки конфигурации (GUI + консоль)
<code>update.py</code>	— утилита обновления программы
<code>schedule_generator.py</code>	— генератор расписания по углу Солнца
<code>cameras/</code>	— пакет драйверов камер
<code>__init__.py</code>	
<code>cannon_driver.py</code>	— Canon DSLR: подключение, настройка, съёмка (gphoto2)
<code>sptt_driver.py</code>	— CSDU-429: USB, съёмка, FITS
<code>sptt_load_firmware.py</code>	— загрузчик прошивки CSDU-429 (FX2 + FPGA)
<code>infra_driver.py</code>	— Tanho SW1300 SWIR: USB, съёмка, TIFF/PNG/FITS
<code>sentry_driver.py</code>	— Princeton CCD via imagerd_rt: супервизор демона
<code>utils.py</code>	— конфигурация, расписание, сеть, системная информация
<code>mqtt_client.py</code>	— MQTT публикация и подписка (консоль + GUI)
<code>worker_common.py</code>	— общая обвязка воркеров: статусы, публикация кадров
<code>frame_archive.py</code>	— чтение архива кадров, конвертация в JPEG, метрика резкости
<code>camera_service.py</code>	— потокобезопасная прослойка между воркером и сетью
<code>frame_server.py</code>	— HTTP-сервер кадров на машине с камерой
<code>discovery.py</code>	— обнаружение камер в локальной сети (UDP)
<code>net_client.py</code>	— HTTP-клиент и Qt-помощники для программ наблюдателя
<code>gui_app.py</code>	— графический интерфейс (вкладки всех камер + Monitor)
<code>monitor.py</code>	— виджет мониторинга (MQTT + локальные файлы статуса)
<code>generate_pdf.py</code>	— генератор PDF документации из README.md
<code>config.json</code>	— конфигурация (не перезаписывается при обновлении)
<code>firmware/</code>	— бинарные файлы прошивки SPTT (fx2, fpga)
<code>infra_lib/</code>	— библиотека libTanhoAPI.so для камеры SW1300
<code>cannon_driver.py</code>	— совместимость (re-export из cameras/)
<code>sptt_driver.py</code>	— совместимость (re-export из cameras/)
<code>infra_driver.py</code>	— совместимость (re-export из cameras/)
<code>sptt_load_firmware.py</code>	— совместимость (re-export из cameras/)
<code>cannon-camera/</code>	— субмодуль (оригинальный репозиторий Canon)
<code>SPTT-CAM/</code>	— субмодуль (оригинальный репозиторий SPTT)
<code>sentry-cam/</code>	— субмодуль (оригинальные программы измерений imagerd_rt)

Установка

```
git clone --recurse-submodules <url>
cd every-camera
pip install -r requirements.txt
```

Зависимости по камерам

Камера	Обязательные пакеты
Canon	`gphoto2-cffi`, `Pillow`, `numpy`
SPTT	`pyusb`, `astropy`, `numpy`
Infra	`numpy`, `opencv-python` или `Pillow`, `astropy` (для FITS)
Sentry	`numpy`, `astropy` (опционально); нужен установленный `imagerd_rt`
Общие	`paho-mqtt` (для MQTT)
GUI	`PyQt5`

Если используется только одна камера — устанавливать зависимости остальных не нужно.

Установка на машину наблюдателя (Windows / Linux)

viewer_app.py и focus_app.py не импортируют драйверы камер — ни gphoto2, ни pyusb, ни libTanhos PI.so. На компьютер наблюдателя достаточно скопировать репозиторий и поставить:

```
pip install PyQt5 numpy Pillow paho-mqtt
pip install astropy # опционально: без него FITS читается встроенным парсером
```

paho-mqtt нужен только для вкладки HiveMQ в просмотрщике; по локальной сети всё работает на стандартной библиотеке.

Быстрый старт

Консольный режим (без дисплея)

```
# Canon DSLR
python main.py --type cannon

# SPTT (CSDU-429)
python main.py --type sptt

# Infra (SW1300 SWIR)
python main.py --type infra

# Sentry (Princeton CCD via imagerd_rt)
python main.py --type sentry
```

Графический режим

```
# Автоматически (если есть дисплей)
python main.py

# Явно
python main.py --gui

# Только одна камера
python main.py --gui --type cannon
python main.py --gui --type sptt
python main.py --gui --type infra
python main.py --gui --type sentry
```

Мониторинг (отдельная программа)

```
python monitor_app.py
```

Программы наблюдателя

```
python viewer_app.py    # просмотр кадров: LAN — любой кадр, HiveMQ — последний
python focus_app.py     # настройка резкости: живой поток (только LAN)
```

Мастер настройки (setup_app.py)

setup_app.py — отдельная программа для настройки конфигурации. Читает и записывает тот же config.json, что использует main.py.

GUI-режим

```
# Все камеры (вкладки Canon / SPTT / Infra / Sentry / MQTT / LAN Server / General)
python setup_app.py

# Только одна камера
python setup_app.py --type sentry

# Другой файл конфигурации
python setup_app.py --config /path/to/config.json
```

Окно мастера содержит вкладки для каждой камеры со всеми настройками. Вкладка **Sentry** включает редактор слотов расписания (таблица с кнопками «Add slot», «Remove selected», «Move up/down»). Вкладка **LAN Server** настраивает HTTP-сервер кадров, к которому подключаются viewer_app.py и focus_app.py.

Кнопки окна: **Save** (сохранить), **Load config...** (загрузить другой файл), **Reset to defaults** (сбросить в заводские значения), **Close**.

Консольный режим

```
# Интерактивная настройка всех камер (выбор в начале)
python setup_app.py --console

# Только одна камера
python setup_app.py --console --type infra

# Другой файл конфигурации
python setup_app.py --console --config /path/to/config.json
```

Консольный мастер задаёт все параметры через серию вопросов в терминале с подсказками и значениями по умолчанию.

> **Примечание.** При первом запуске main.py в консольном режиме, если конфигурация пуста, программа автоматически запустит мастер настройки для выбранной камеры. setup_app.py — это то же самое, но без привязки к режиму измерений.

Конфигурация

Файл config.json в корне проекта. Создаётся автоматически при первом запуске.

```
{
```

```
"cannon": {
  "instance_name": "",
  "output_dir": "/path/to/output",
  "schedule_file": "/path/to/schedule.txt",
  "capture_seconds": [0, 30],
  "camcfg_file": ""
},
"sptt": {
  "instance_name": "",
  "output_dir": "/path/to/output",
  "exposure": 0.88,
  "gain": 100,
  "binning": 0,
  "encoding": 1,
  "target_temp": null,
  "firmware_dir": "",
  "capture_seconds": [0, 30]
},
"infra": {
  "instance_name": "",
  "output_dir": "/path/to/output",
  "schedule_file": "/path/to/schedule.txt",
  "capture_seconds": [0, 30],
  "exposure_us": 1000.0,
  "gain": 0,
  "roi": "1280x1024",
  "save_format": "tiff"
},
"sentry": {
  "instance_name": "",
  "imagerd_rt_dir": "/usr/local/imagerd_rt",
  "output_dir": "/path/to/fits/archive",
  "device_id": "ASI1",
  "site_id": "SITE",
  "zenith_angle_start": -10,
  "schedule_len_ms": 1440000,
  "imaging_mode": "schedule",
  "capture_seconds": [0, 30],
  "slots": [
    {
      "filter_num": 1,
      "start_ms": 0,
      "exposure_ms": 55000,
      "binning": 2,
      "gain": 3,
      "readout_ms": 5000
    }
  ]
},
"mqtt": {
  "enabled": false,
  "host": "broker.hivemq.com",
  "port": 1883,
  "user": "",
  "password": "",
  "prefix": "every_camera",
  "tls": false
},
"server": {
  "enabled": true,
  "bind": "0.0.0.0",
  "port": 8765,
  "discovery": true,
  "discovery_port": 45455,
  "max_list": 2000,
  "focus_ttl": 60
},
"status_dir": ""
```

```
}
```

Параметры

Canon (`cannon`)

- **instance_name** — имя экземпляра (если пусто — генерируется автоматически: `Cannon\_<IP>`)
- **output_dir** — директория для сохранения JPEG снимков
- **schedule_file** — путь к файлу расписания
- **capture_seconds** — секунды каждой минуты для съёмки (по умолчанию `[0, 30]`)
- **camcfg_file** — путь к .ini файлу настроек камеры (генерируется автоматически при первом запуске)

SPTT (`sptt`)

- **instance_name** — имя экземпляра (если пусто — `SPTT\_<IP>`)
- **output_dir** — директория для сохранения FITS файлов
- **exposure** — экспозиция в секундах (по умолчанию `0.88`)
- **gain** — усиление (0-1023, по умолчанию `100`)
- **binning** — биннинг (`0` = 1×1, `1` = 2×2, `3` = 4×4)
- **encoding** — разрядность (`0` = 8 бит, `1` = 12 бит)
- **target_temp** — целевая температура ПЗС в °C (`null` = охлаждение не задано)
- **firmware_dir** — директория с файлами прошивки (если пусто — используется `firmware/` рядом со скриптом)
- **capture_seconds** — секунды каждой минуты для публикации в MQTT

Infra (`infra`)

- **instance_name** — имя экземпляра (если пусто — `Infra\_<IP>`)
- **output_dir** — директория для сохранения снимков
- **schedule_file** — путь к файлу расписания
- **capture_seconds** — секунды каждой минуты для съёмки (по умолчанию `[0, 30]`)
- **exposure_us** — экспозиция в микросекундах (по умолчанию `1000.0`)
- **gain** — усиление (0-120, по умолчанию `0`)
- **roi** — область захвата (`"1280x1024"` — полный кадр, `"1280x256"` — горизонтальная полоса)
- **save_format** — формат сохранения: `"tiff"` (16-bit raw), `"png"` (8-bit нормализованный), `"fits"` (FITS с метаданными)

Камера Tanho THCAMSW1300 (сенсор Sony IMX990-AABA-C), SWIR-диапазон, 12-bit ADC.

Sentry (`sentry`)

Камера Princeton Instruments CCD управляется через сторонний демон `imagerd_rt` (Keo Scientific, бинарный файл). Драйвер `every-camera` выступает **супервизором**: генерирует `schedule.conf`, запускает демон, читает его выходные файлы и публикует статус/кадры через MQTT.

- **instance_name** — имя экземпляра (если пусто — `Sentry\_<IP>`)
- **imagerd_rt_dir** — директория установки `imagerd\_rt` (по умолчанию `/usr/local/imagerd\_rt`)
- **output_dir** — директория для архива FITS файлов (опционально)
- **device_id** — идентификатор устройства камеры (например `"ASI1"`)
- **site_id** — идентификатор станции (используется в именах файлов)
- **zenith_angle_start** — зенитный угол Солнца, при котором начинаются измерения (градусы;

отрицательное = Солнце ниже горизонта, по умолчанию -10°)

- schedule_len_ms** — длительность одного цикла расписания в миллисекундах (по умолчанию 1440000 = 24 мин)
- imaging_mode** — режим съёмки: "schedule" (по слотам расписания) или "rapid" (непрерывная съёмка)
- capture_seconds** — секунды каждой минуты для опроса статуса демона / публикации в MQTT
- slots** — список слотов расписания (см. ниже)

Слоты расписания Sentry

Каждый слот описывает один сеанс съёмки с определённым светофильтром внутри цикла.
Параметры слота:

Поле	Описание
filter_num	Номер светофильтра
start_ms	Время начала слота от начала цикла (мс)
exposure_ms	Длительность экспозиции (мс)
binning	Биннинг: 1, 2 или 4
gain	Усиление ПЗС (1-3 для ProEM; допустимые значения зависят от модели)
readout_ms	Время перемещения колеса фильтров + считывания матрицы (мс)

Пример — два слота, фильтры 1 и 2, каждые 24 секунды:

```
"slots": [
  { "filter_num": 1, "start_ms": 0,      "exposure_ms": 8000, "binning": 2, "gain": 3, "readout_ms": 3000 },
  { "filter_num": 2, "start_ms": 12000, "exposure_ms": 8000, "binning": 2, "gain": 3, "readout_ms": 3000 }
]
```

Из этих слотов драйвер автоматически генерирует файл schedule.conf в формате imagerd_rt при каждом запуске.

Файловый интерфейс imagerd_rt

Файл	Описание
<dir>/schedule.conf	Расписание (генерируется драйвером)
<dir>/aux/image.fits	Последний захваченный кадр
<dir>/aux/meta-data.txt	Метаданные текущего кадра (пары ключ-значение, TAB)
<dir>/aux/seqno.txt	Счётчик кадров
<dir>/aux/stop_file	Управление: запись "10" = стоп, "11" = возобновить

MQTT (mqtt)

- enabled** — включить/выключить MQTT публикацию
- host** — адрес брокера (по умолчанию broker.hivemq.com)
- port** — порт (1883 стандартный, 8883 с TLS)
- user** — имя пользователя (опционально)
- password** — пароль (хранится открытым текстом в config.json)
- prefix** — префикс топиков (по умолчанию every_camera)
- tls** — использовать TLS-соединение

Структура топиков:

```
{prefix}/{instance_name}/status — статус камеры (retain=true)
{prefix}/{instance_name}/frame  — последний кадр (JPEG, base64)
{prefix}/{instance_name}/cmd/#  — команды (get_frame, capture_frame)
```

При штатной остановке камера очищает retained-статус, а на случай аварийного обрыва

регистрируется MQTT last will со статусом offline. Поэтому исчезнувшая камера сразу пропадает из монитора, а не висит в нём как «running» бесконечно.

LAN-сервер кадров (`server`)

- **enabled** — включить HTTP-сервер кадров на машине с камерой
- **bind** — адрес прослушивания (`0.0.0.0` — вся локальная сеть, `127.0.0.1` — только эта машина)
- **port** — порт HTTP (по умолчанию `8765`)
- **discovery** — отвечать на широковещательные UDP-запросы поиска камер
- **discovery_port** — порт обнаружения (по умолчанию `45455`)
- **max_list** — максимальное число кадров в одном ответе на листинг архива
- **focus_ttl** — сколько секунд камера остаётся в режиме фокусировки после последнего запроса от focus_app.py (по умолчанию 60)

> **Доступ не защищён паролем.** Любой в локальной сети может смотреть кадры и, если камера это поддерживает, менять параметры съёмки. Используйте сервер только в доверенной сети; чтобы закрыть доступ, поставьте "bind": "127.0.0.1" или "enabled": false.

Общие

- **status_dir** — директория для JSON-файлов статуса (по умолчанию `~/every\_camera/status`)

Файл расписания (Canon и Infra)

Формат: одна строка = один интервал измерений.

```
# Measurement Schedule
# Format: %Y-%m-%d %H:%M:%S - %Y-%m-%d %H:%M:%S

2026-03-08 12:00:00 - 2026-03-08 23:59:00
2026-03-09 08:00:00 - 2026-03-09 18:00:00
```

- Строки с # — комментарии
- Камера снимает только когда текущее время попадает в один из интервалов
- Вне интервалов программа ждёт (статус waiting)

> **Sentry** расписание задаётся иначе — через slots в config.json. Файл schedule.conf для imagerd_rt генерируется автоматически.

Режим предпросмотра (preview)

```
# Infra: непрерывная запись preview_Infra.fits/.tiff на максимальной частоте
python main.py --type infra --preview

# Canon: то же для preview_Canon.jpg
python main.py --type cannon --preview

# Sentry: копирование image.fits → preview_Sentry.fits при каждом новом кадре
python main.py --type sentry --preview
```

В графическом режиме вкладка **Infra Camera** имеет встроенный **Live Preview** для настройки резкости и параметров камеры в реальном времени:

- 1. Нажмите **Connect** для подключения к камере
- 2. Нажмите **Live Preview** для запуска потока видео
- 3. Настройте **Exposure, Gain** и **ROI** — изменения применяются немедленно
- 4. Нажмите **Stop Preview** когда настройка завершена

Live Preview и режим измерений по расписанию работают независимо.

Мониторинг

```
python monitor_app.py
```

Программа monitor_app.py показывает состояние всех запущенных экземпляров камер:

- **Вкладка MQTT** — подключается к MQTT брокеру, отображает статусы в реальном времени
- **Вкладка Local files** — читает JSON-файлы из директории `~/every_camera/status``

Таблица показывает: имя экземпляра, тип камеры, PID, статус, количество снимков, время последнего снимка, количество ошибок, доп. информацию (ISO, экспозиция, температура, свободное место на диске).

Статусы

Статус	Цвет	Описание
`RUNNING`	зелёный	Камера активно снимает
`WAITING`	оранжевый	Ожидание расписания
`ERROR`	красный	Ошибка при съёмке
`STOPPED`	серый	Остановлена
`OFFLINE`	оранжевый	Связь с камерой оборвалась (MQTT last will)
`STALE`	оранжевый	Нет обновлений более 30 секунд

Во вкладке **Local files** записи процессов, которых больше нет в системе, помечаются как STOPPED и (dead) — раньше упавший процесс висел в списке как работающий.

Просмотр последнего кадра

В MQTT-вкладке монитора доступна кнопка **View Last Frame**:

- 1. Подключитесь к MQTT брокеру
- 2. Выберите экземпляр камеры в таблице
- 3. Нажмите **View Last Frame**

Монитор отправит команду камере через MQTT. Камера ответит последним снятым кадром (JPEG, base64). Трафик расходуется только по запросу.

Просмотрщик кадров (`viewer\_app.py`)

Программа наблюдателя: только просмотр, ничего не снимает и не меняет настроек камеры. Работает на Windows и Linux, драйверы камер не нужны.

```
python viewer_app.py           # найти камеры в локальной сети
python viewer_app.py --host 192.168.1.5 # сразу подключиться к камере
python viewer_app.py --mqtt      # открыть сразу вкладку HiveMQ
```

Вкладка «Local network»

Полный доступ к архиву камеры:

1. **Search LAN** — широковещательный запрос, в списке появляются все найденные камеры. Если обнаружение выключено или заблокировано, введите хост:порт вручную.
2. Выпадающий список дат и список кадров — можно открыть **любой** снятый кадр, хоть месячной давности.
3. **Latest frame** — последний кадр; **Follow live** обновляет его каждые 3 секунды.
4. **Contrast** — растяжка гистограммы для 16-битных кадров: min-max, перцентильная (0.5–99.5 %, лучше видно слабые детали рядом с яркими пикселями) или без растяжки.
5. **Save original...** — скачать исходный файл (FITS/TIFF/PNG/JPEG) без преобразований.

Под изображением показываются реальные характеристики кадра: размер, тип данных, минимум/максимум/среднее, доля насыщенных пикселей и ключи FITS-заголовка.

Архив читается прямо с диска камеры, поэтому кадры доступны, даже если камера сейчас занята измерениями или её рабочий поток завис.

Вкладка «HiveMQ»

Для камер, доступных только через интернет. Здесь запрашивается **только последний кадр** — архив по MQTT не листается, чтобы не гонять мегабайты base64 через публичный брокер.

1. Заполните параметры брокера и нажмите **Connect**.
2. В таблице появятся камеры, публикующие статус.
3. Выберите камеру и нажмите **Request latest frame**.

Настройка резкости (`focus\_app.py`)

Помощник наведения резкости: живой поток кадров, изменяемые параметры съёмки и числовая оценка резкости. **Работает только по локальной сети** — живой поток через публичный брокер потребовал бы неприемлемого трафика.

```
python focus_app.py                # найти камеры в локальной сети
python focus_app.py --host 192.168.1.5 # сразу подключиться
```

Порядок работы:

1. **Search LAN** → выбрать камеру → **Start live view**.
2. Крутить фокусер, наблюдая за числом резкости и графиком: при точной фокусировке значение достигает максимума (100 % of best), а зелёная пунктирная линия показывает лучший результат за сеанс.
3. Мышью можно выделить область кадра — резкость будет считаться только по ней (удобно наводиться по одной звезде). Кнопка **Whole frame** возвращает весь кадр.
4. Панель **Camera parameters** строится по ответу самой камеры, поэтому у каждой камеры свои поля. Изменить значение → **Apply**.
5. **Stop** возвращает камеру в обычный режим.

Строка под графиком предупреждает о пересвете ($\triangle$ overexposed), если насыщено больше 1 % пикселей.

Камера	Живой поток	Изменяемые параметры
Canon	Live View (`get_preview`), при отказе — обрыв потока	`shutter_speed`, `aperture`, `whitebalance`
SPTT	непрерывный захват между плановыми снимками	`exposure`, `gain`, `binning`, `encoding`
Infra	поток кадров, который камера и так подготавливает	`capture_us`, `gain`, `roi`
Sentry	кадры демона `imagerd_rt` по мере появления	его расписание задаётся в `schedule.conf` и тр

Почему это не мешает измерениям

- Плановая съёмка всегда в приоритете: рядом с секундами расписания (capture_seconds) кадры для фокусировки не берутся, поэтому снимок не может задержаться.
- Камера снимает в свободном режиме, только пока focus_app.py подтверждает, что за ней смотрят. У запроса есть срок (server.focus_ttl, по умолчанию 60 с): закрытие окна, обрыв сети или аварийное завершение программы сами возвращают камеру к работе по расписанию.
- Пять ошибок подряд при съёмке кадров фокусировки — режим выключается сам.

HTTP-сервер кадров и обнаружение в сети

На каждой машине с камерой поднимается небольшой HTTP-сервер (frame_server.py). Он запускается автоматически вместе с измерениями — и в консольном режиме, и в GUI.

Сервер устроен так, чтобы никогда не мешать измерениям:

- работает в отдельных daemon-потоках; при занятом порте выводит предупреждение и измерения продолжаютс как ни в чём не бывало;
- обработчики запросов **не обращаются к камере** — живые кадры берутся из потокобезопасной прослойки, архив читается с диска;
- ограничено число одновременных видеопотоков и размер листинга;
- имя файла проверяется на выход за пределы директории архива.

Точки доступа

Запрос	Назначение
`GET /`	Встроенная веб-страница — можно смотреть кадры прямо из браузера
`GET /api/info`	Имя экземпляра, тип камеры, возможности
`GET /api/status`	Текущий статус воркера
`GET /api/dates`	Даты, за которые есть кадры
`GET /api/frames?date=&limit=&offset=`	Листинг архива
`GET /api/frame?name=&as=jpeg\raw&max=&stretch=`	Кадр
`GET /api/thumb?name=`	Миниатюра
`GET /api/stats?name=`	Статистика и гистограмма кадра (без `name` — живого)
`GET /api/latest.jpg`	Последний живой кадр
`GET /api/live.mjpg?fps=&max=`	MJPEG-поток (используется `focus_app.py`)
`GET /api/params`	Текущие параметры и схема редактируемых полей
`POST /api/params`	Запрос на изменение параметров
`POST /api/focus`	Включить/продлить/выключить режим фокусировки

Просмотр архива без камеры

Сервер можно запустить отдельно, просто над директорией с кадрами — например, на

машине, куда снимки скопированы:

```
python -m frame_server --output-dir /path/to/output --port 8765
```

Дальше к нему подключается viewer_app.py или обычный браузер.

Обнаружение камер

```
python discovery.py --timeout 2
```

Рассылает широковещательный UDP-запрос и печатает все ответившие камеры с их адресами. Тот же механизм использует кнопка **Search LAN**. Если сеть блокирует широковещательные пакеты, адрес всегда можно ввести вручную.

Генератор расписания по углу Солнца

Утилита schedule_generator.py автоматически рассчитывает время захода/восхода Солнца через заданный угол высоты и создаёт файл расписания для камер Canon и Infra.

Использование

```
# Интерактивный режим (выбор предустановленной точки наблюдения)
python schedule_generator.py

# С параметрами командной строки
python schedule_generator.py --lat 51.81 --lon 103.08 --alt 680 \
    --angle -10 --start 2026-01-01 --days 100 --output schedule.txt

# С предустановленной точкой
python schedule_generator.py --site GPHO_TK --angle -10 --start 2026-01-01 --days 100 -o schedule.txt

# Графический режим
python schedule_generator.py --gui
```

Параметры

Параметр	Описание
`--lat`	Широта (градусы)
`--lon`	Долгота (градусы)
`--alt`	Высота над уровнем моря (метры)
`--angle`	Пороговый угол высоты Солнца (градусы, например `-10`)
`--start`	Начальная дата (`YYYY-MM-DD`)
`--days`	Количество дней для расчёта
`--output` / `-o`	Путь к выходному файлу расписания
`--site`	Предустановленная точка наблюдения
`--gui`	Запустить в графическом режиме

Предустановленные точки наблюдения

Код	Название	Широта	Долгота	Высота
`GPHO_TK`	GPHO Tory	51.8106°	103.0755°	680 м
`KLNG_1`	Kaliningrad	54.6033°	20.2111°	30 м
`MAGNITKA`	Magnitka	55.9305°	48.7449°	95 м
`SURA_ISZF`	SURA ISZF	56.1471°	46.0985°	172 м
`SURA_NNGU`	SURA NNGU	56.1502°	46.1050°	184 м

`SURA`	SURA	56.1437°	46.0990°	173 м
`PEREVOZ`	Perevoz	55.5426°	44.5357°	155 м

Зависимости: astropy (входит в требования SPTT-камеры).

Обновление

Утилита update.py обновляет программу до последней версии из git-репозитория.

Обновляются только скрипты (.py), директории cameras/, firmware/, infra_lib/ и requirements.txt.

Конфигурация (config.json, camcfg_*.ini) и расписания (*schedule*.txt) не затрагиваются.

```
# Обновить до последней версии
python update.py

# Посмотреть что изменится (без применения)
python update.py --dry-run

# Перезаписать даже локально изменённые файлы
python update.py --force
```

Остановка

- Консольный режим: Ctrl+C — программа корректно останавливает все потоки
- GUI: закрыть окно — конфигурация сохраняется автоматически (в тот файл, который был загружен через --config), MQTT отключается
- Sentry: при остановке драйвер посылает сигнал stop_file=10 демону imagerd_rt; демон завершит текущий цикл и остановится
- При остановке снимается retained-статус в MQTT и удаляется файл статуса, поэтому камера сразу исчезает из монитора и просмотрщика
- LAN-сервер кадров останавливается вместе с измерениями: после этого архив по сети недоступен, пока камера не будет запущена снова (или пока не будет запущен `python -m frame_server --output-dir ...` отдельно)

USB-права (Linux)

Для работы с камерами без sudo:

Canon (gphoto2)

```
# Обычно работает из коробки. Если нет — убить gvfs:
sudo pkill gvfsd-gphoto2
```

SPTT (CSDU-429)

```
echo 'SUBSYSTEM=="usb", ATTR{idVendor}=="04b4", MODE="0666" ' \
| sudo tee /etc/udev/rules.d/99-sptt.rules
sudo udevadm control --reload-rules
```

Infra (SW1300)

```
echo 'SUBSYSTEM=="usb", ATTR{idVendor}=="aa55", ATTR{idProduct}=="8866", MODE="0666", GROUP="plugdev" ' \
```

```
| sudo tee /etc/udev/rules.d/99-sw1300-camera.rules  
sudo udevadm control --reload-rules && sudo udevadm trigger
```

Sentry (imagerd_rt)

Демон `imagerd_rt` обращается к камере напрямую через SDK (Picam + Pleora eBUS). Права настраиваются согласно документации Keo Scientific / Princeton Instruments для конкретной модели. Every-camera запускает `imagerd_rt_start` как пользовательский процесс — для этого достаточно прав на исполнение бинарного файла.

Генерация PDF документации

```
pip install fpdf2  
python generate_pdf.py  
# → README.pdf
```